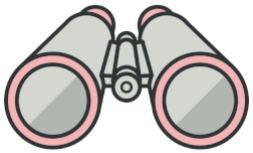
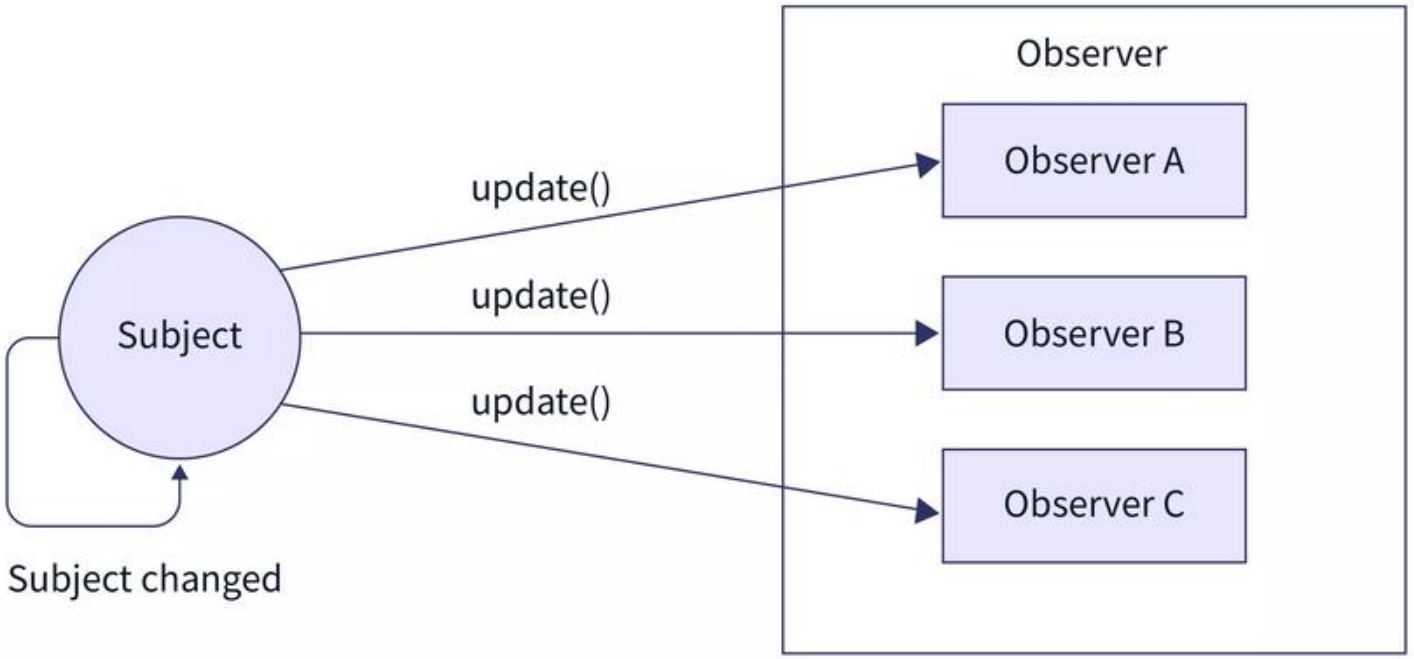
Поведенческие паттерны. Наблюдатель

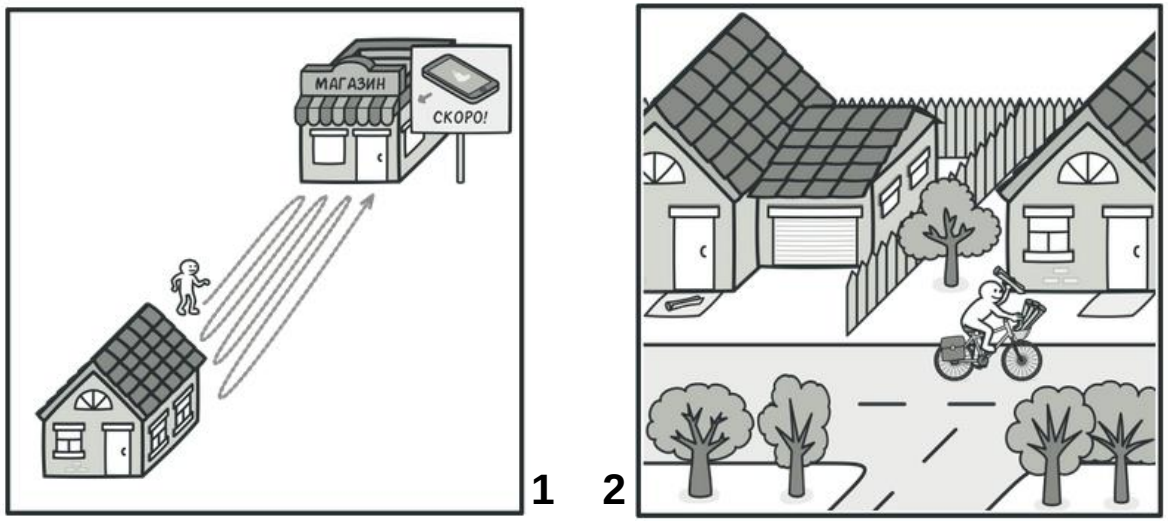


Паттерн Наблюдатель
Observer (Издатель-Подписчик)



Наблюдатель — это поведенческий паттерн проектирования, который создаёт механизм подписки, позволяющий одним объектам следить и реагировать на события, происходящие в других объектах.
При этом наблюдатели могут свободно подписываться и отписываться от этих оповещений.

Уведомление об изменении состояния объекта



Аналогия

После того как *подписчик* подписался на журнал, подписчик ожидает пока *издатель издаст* журнал и *оповестит* об этом подписчика.

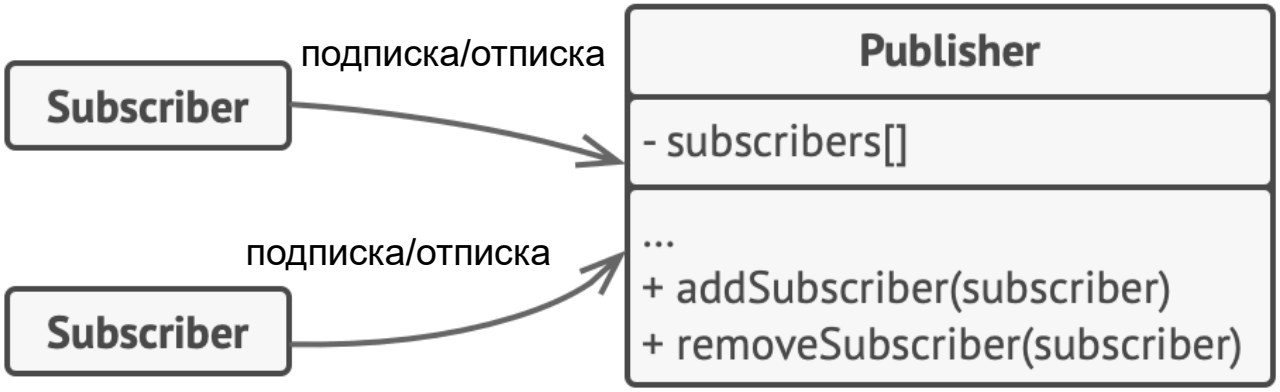
Имеется два способа получения подписчиком журнала:

Первый способ - «*метод вытягивания*» (pool): После получения уведомления от издателя о том, что журнал выпущен, подписчик должен пойти к издателю и забрать (*вытянуть*) журнал самостоятельно.

Второй способ – «*метод проталкивания*» (push): Издатель не уведомляет подписчика о выпуске журнала, а самостоятельно или через почту доставляет журнал подписчику и бросает (*проталкивает*) журнал в почтовый ящик.

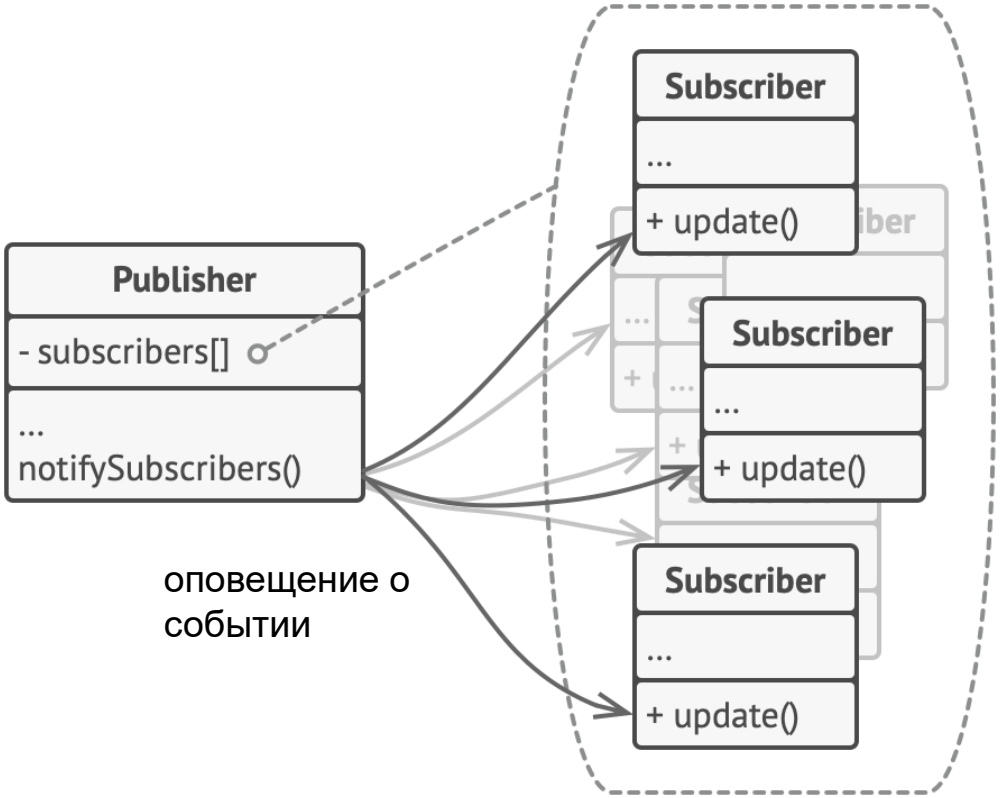
Решение

Паттерн **Наблюдатель** – использует связь отношения зависимости «один ко многим» (один *издатель* ко многим *подписчикам*). При изменении состояния одного объекта (*издателя*), все зависящие от него объекты (*подписчики*) оповещаются об этом и автоматически обновляются.



Паттерн **Наблюдатель** предлагает хранить внутри объекта издателя список ссылок на объекты подписчиков, причём издатель не должен вести список подписки самостоятельно. Он предоставит методы, с помощью которых подписчики могли бы добавлять или убирать себя из списка.

Решение



Когда в *издателе* будет происходить отслеживаемое событие, он будет просматривать список *подписчиков* и оповещать их об этом, вызывая определённый метод объектов-подписчиков.

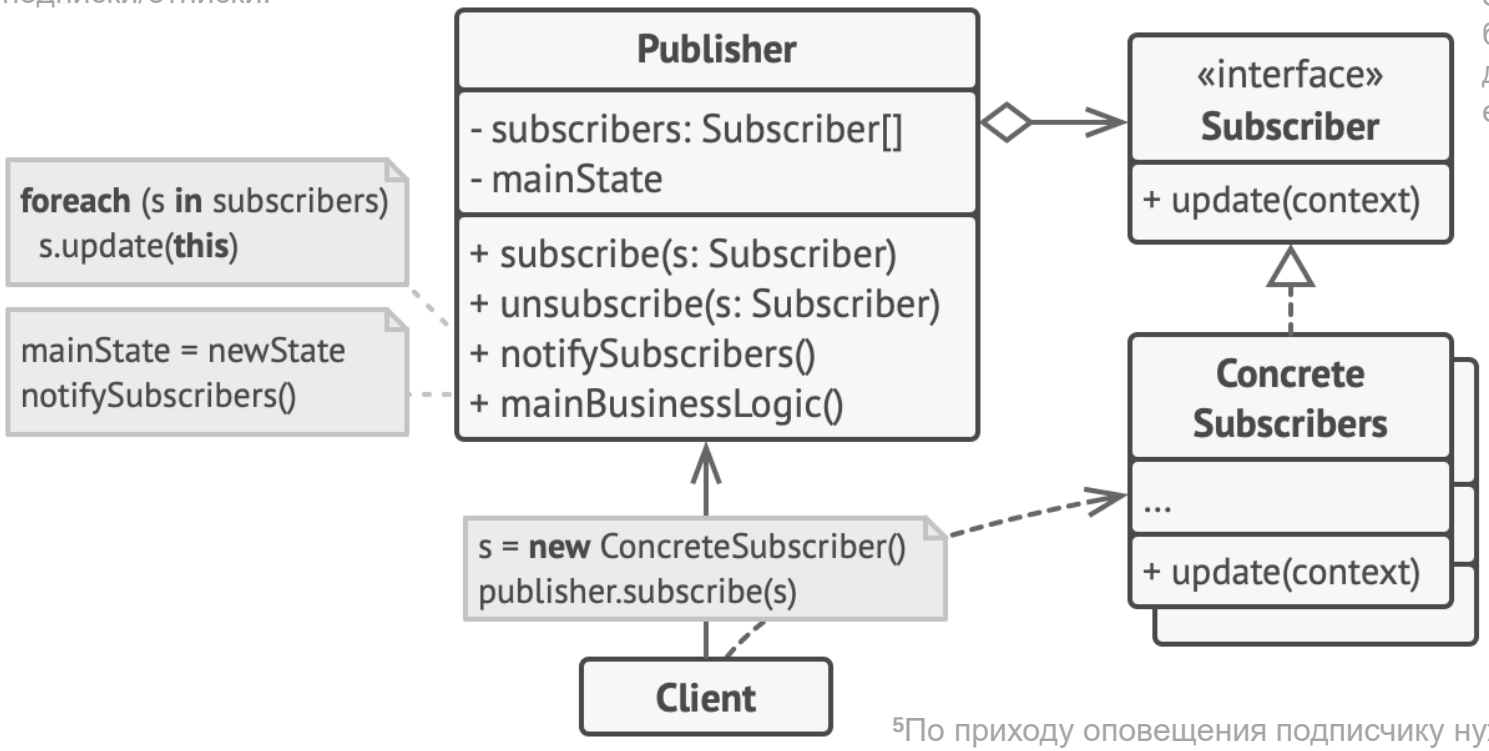
Издателю безразлично, какой класс будет иметь тот или иной *подписчик*, так как все они должны следовать общему интерфейсу и иметь единый метод оповещения.

Структура паттерна

[1]**Издатель** характеризуется внутренним состоянием, изменение которого интересно отслеживать подписчикам. Издатель содержит механизм подписки: список подписчиков и методы подписки/отписки.

[2]Когда внутреннее состояние издателя меняется, он оповещает своих подписчиков. Для этого издатель проходит по списку подписчиков и вызывает их метод оповещения, заданный в общем интерфейсе подписчиков.

[3]**Подписчик** определяет интерфейс, которым пользуется издатель для отправки оповещения. В большинстве случаев для этого достаточно единственного метода.



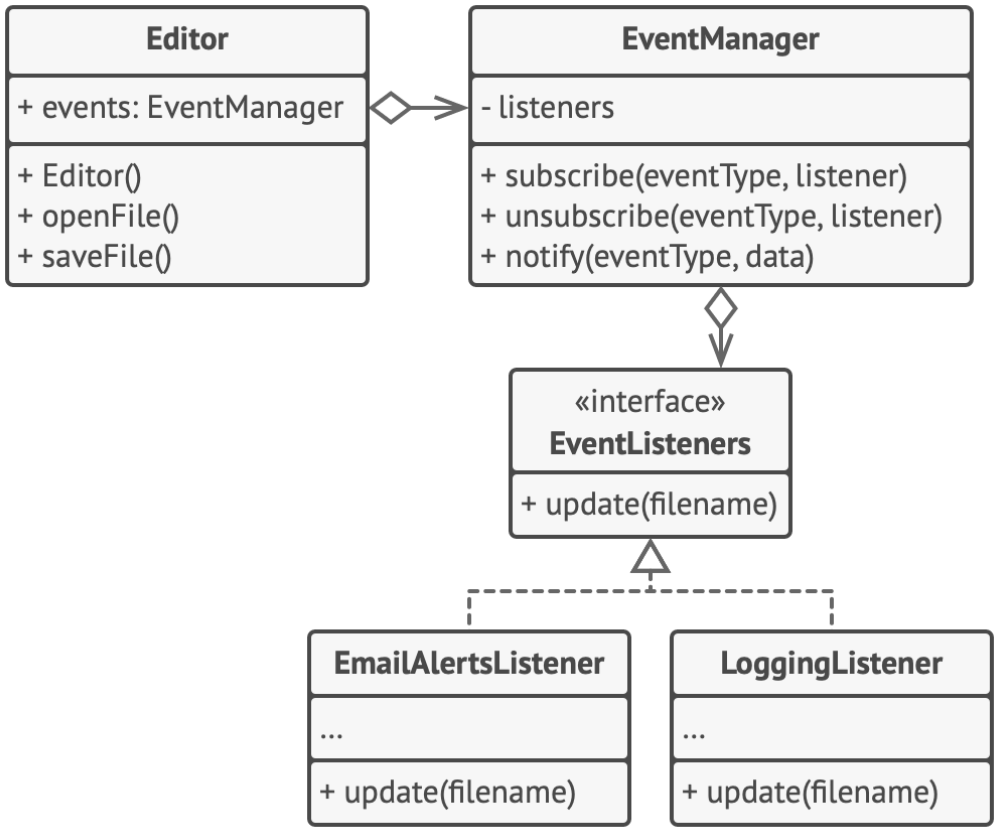
[4]**Конкретные подписчики** выполняют что-то в ответ на оповещение, пришедшее от издателя. Эти классы должны следовать общему интерфейсу подписчиков, чтобы издатель не зависел от конкретных классов подписчиков.

[5]По приходу оповещения подписчику нужно получить обновлённое состояние издателя. Издатель может передать это состояние через параметры метода оповещения. Более гибкий вариант — передавать через параметры весь объект издателя, чтобы подписчик мог сам получить требуемые данные. Как вариант, подписчик может постоянно хранить ссылку на объект издателя, переданный ему в конструкторе.

[6]**Клиент** создаёт объекты издателей и подписчиков, а затем регистрирует подписчиков на обновления в издателях.

Пример

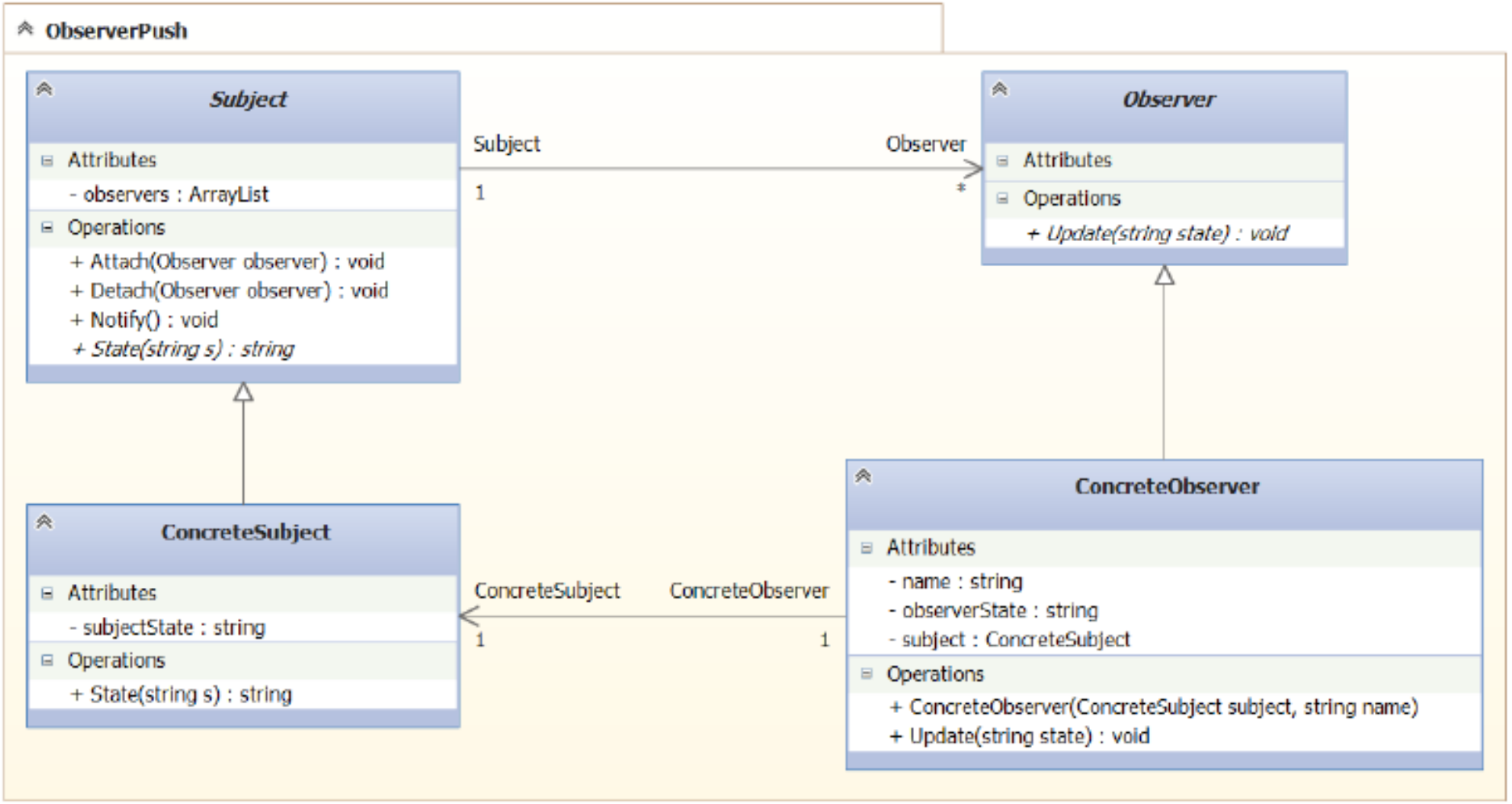
Оповещение объектов о событиях в других объектах.



- Список подписчиков меняется динамически: объекты могут как подписываться на определённые события, так и отписываться от них во время выполнения программы.
- Редактор не ведёт список подписчиков самостоятельно, а делегирует это вложенному объекту. Это даёт возможность использовать механизм подписки не только в классе редактора, но и в других классах программы.
- Для добавления в программу новых подписчиков не нужно менять классы издателей, пока они работают с подписчиками через общий интерфейс.

Структура паттерна UML

Модель проталкивания Push/Pull model



Структура паттерна на языке C#

Модель проталкивания Push/Pull model

`Subject` - **Субъект** *(издатель)*:
Издатель содержит ссылки на своих подписчиков и предоставляет интерфейс (набор методов) для добавления и удаления подписчиков. На издателя может ссылаться любое число подписчиков.

```
class Program
{
    static void Main()
    {
        ConcreteSubject subject = new ConcreteSubject();
        subject.Attach(new ConcreteObserver(subject));
        subject.Attach(new ConcreteObserver(subject));
        subject.State = "Some State ...";
        subject.Notify();
    }
}
```

`Observer` - **Наблюдатель** *(подписчик)*:
Подписчик предоставляет интерфейс (набор методов) для обновления своего состояния при изменении состояния издателя.

`ConcreteObserver` - **Конкретный наблюдатель** *(конкретный подписчик)*:
Реализует интерфейс обновления, предоставляемый абстрактным классом `Observer` и поддерживает согласованность состояния с издателем.

```
abstract class Subject
{
    ArrayList observers = new ArrayList();

    public void Attach(Observer observer)
    {
        observers.Add(observer);
    }

    public void Detach(Observer observer)
    {
        observers.Remove(observer);
    }

    public void Notify()
    {
        foreach (Observer observer in observers)
            observer.Update();
    }
}
```

```
abstract class Observer
{
    public abstract void Update();
}
```

`ConcreteSubject` - **Конкретный субъект** *(конкретный издатель)*:
Конкретный издатель посылает уведомление своим подписчикам и передает им свое состояние.

```
class ConcreteSubject : Subject
{
    public string State { get; set; }
}
```

```
class ConcreteObserver : Observer
{
    string observerState;
    ConcreteSubject subject;

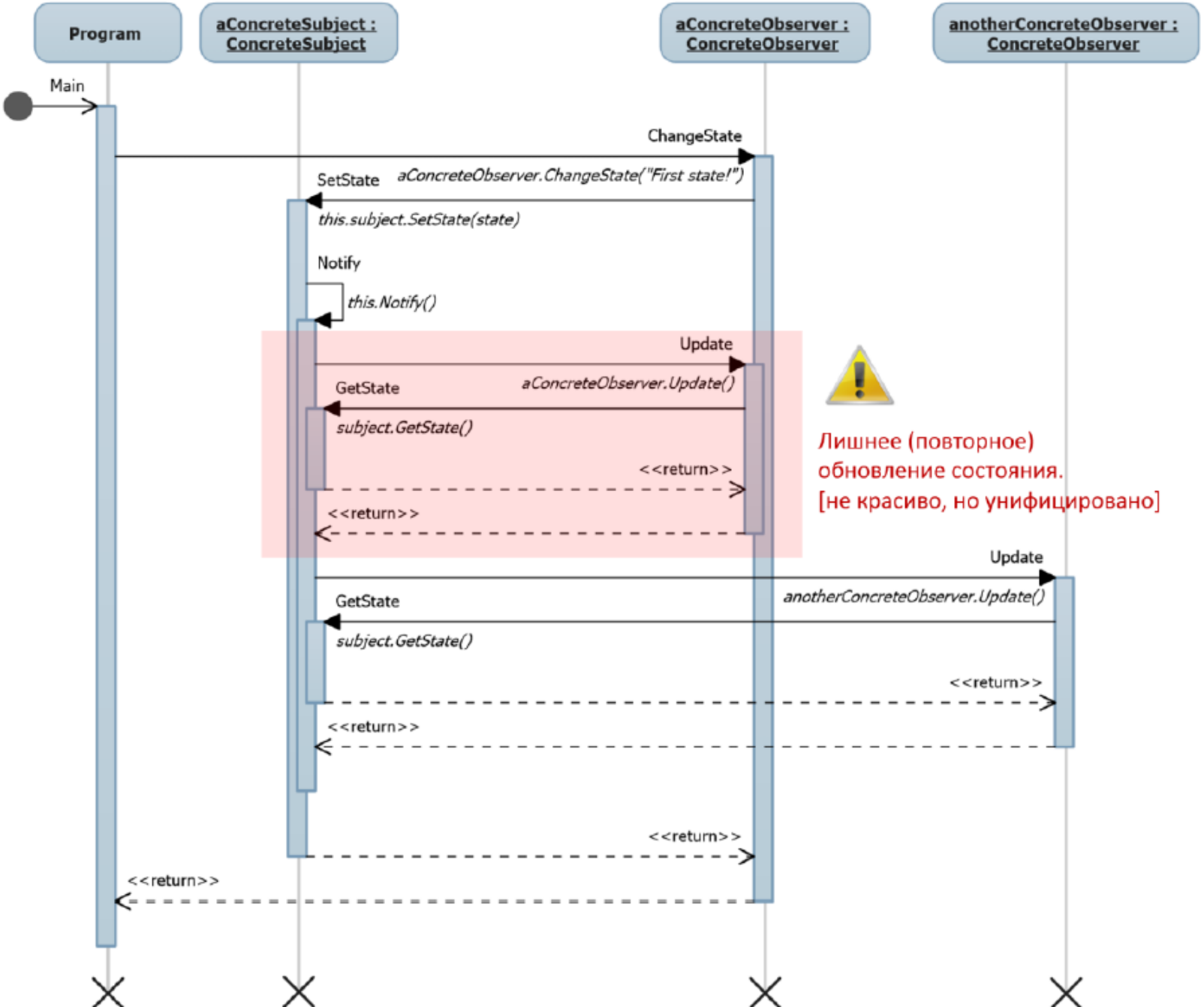
    public ConcreteObserver(ConcreteSubject subject)
    {
        this.subject = subject;
    }

    public override void Update()
    {
        observerState = subject.State;
    }
}
```

Отношения между участниками

Отношения между классами

- Абстрактный класс Subject связан связью отношения ассоциации с абстрактным классом Observer.
- Конкретный класс ConcreteSubject связан связью отношения наследования с абстрактным классом Subject.
- Конкретный класс ConcreteObserver связан связью отношения наследования с абстрактным классом Observer и связью отношения ассоциации с конкретным классом ConcreteSubject..
- Отношения между объектами
- ConcreteSubject (*издатель*) уведомляет своих наблюдателей (подписчиков) о любом изменении, которое могло бы привести к рассогласованности состояний издателя и подписчика.
- ConcreteObserver (*подписчик*) после получения от ConcreteSubject (издателя) уведомления об изменении состояния, может запросить у издателя дополнительную информацию для полного согласования своего состояния.
- Метод Notify класса ConcreteSubject (издателя) может быть вызван не только издателем самостоятельно, также этот метод могут вызывать и подписчики, и посторонние клиенты.


Program
aConcreteSubject : ConcreteSubject
aConcreteObserver : ConcreteObserver
anotherConcreteObserver : ConcreteObserver
Main
ChangeState
SetState
aConcreteObserver.ChangeState("First state!")
this.subject.SetState(state)
Notify
this.Notify()
Update
GetState
aConcreteObserver.Update()
subject.GetState()
<<return>>
<<return>>
Лишнее (повторное) обновление состояния. [не красиво, но унифицировано]
Update
GetState
anotherConcreteObserver.Update()
subject.GetState()
<<return>>
<<return>>
<<return>>
<<return>>

Применение

1. Когда после изменения состояния одного объекта требуется что-то сделать в других, но заранее неизвестно, какие именно объекты должны отреагировать.

Описанная проблема может возникнуть при разработке библиотек пользовательского интерфейса, когда необходимо дать возможность сторонним классам реагировать на клики по кнопкам.

Паттерн *Наблюдатель* позволяет любому объекту с интерфейсом подписчика зарегистрироваться на получение оповещений о событиях, происходящих в объектах-издателях.

2. Когда одни объекты должны наблюдать за другими, но только в определённых случаях.

Издатели ведут динамические списки. Все *наблюдатели* могут подписываться или отписываться от получения оповещений во время выполнения программы.

Шаги реализации

1. **Разбить функциональность приложения** на две части: независимое ядро и опциональные зависимые части. Независимое ядро станет *издателем*. Зависимые части станут *подписчиками*.

2. **Создать интерфейс подписчиков**. Обычно в нём достаточно определить единственный метод оповещения.

3. **Создать интерфейс издателей и описать в нём операции управления подпиской**. Издатель должен работать только с общим интерфейсом подписчиков.

4. **Код управления подпиской вынести в промежуточный абстрактный класс**, от которого будут наследоваться все издатели.

5. **Создать классы конкретных издателей**. Реализовать их так, чтобы после каждого изменения состояния они отправляли оповещения всем своим подписчикам.

6. **Реализовать метод оповещения в конкретных подписчиках**. Предусмотреть параметры, через которые издатель мог бы отправлять какие-то данные, связанные с происшедшим событием.

   Возможен и другой вариант, когда подписчик, получив оповещение, сам возьмёт из объекта издателя нужные данные. Но в этом случае необходимо привязать класс подписчика к конкретному классу издателя.

7. **Клиент должен создавать необходимое количество объектов подписчиков** и подписывать их у издателей.

Преимущества и недостатки

+ Издатели не зависят от конкретных классов подписчиков и наоборот.
+ Можно подписывать и отписывать получателей динамически.
+ Реализует принцип открытости/закрытости.
- Подписчики оповещаются в случайном порядке.